

Roteiro de Apresentação

Checkpoint 1 — Aluno 1

Sistema P2P Híbrido para Compartilhamento de Documentos • Arquivos sob sua responsabilidade: network.c, protocol.c, peer.c

Parte 1 — O que os dois precisam saber

Esta parte é igual nos dois roteiros. Combinem antes quem fala: a sugestão é quem apresentar primeiro cobrir os itens 1.1 a 1.4, e os dois participarem da demonstração.

1.1 O trabalho em uma frase

Um sistema de compartilhamento de documentos entre computadores, escrito em C, no modelo P2P híbrido. Híbrido quer dizer que não é todo mundo falando com todo mundo, nem tudo passando por uma máquina só: alguns nós têm mais responsabilidade — os Super Peers, que sabem quem está na rede — e os Peers comuns se registram neles.

O trabalho inteiro tem 6 checkpoints e termina em 06/11, com DHT Chord, replicação, eleição de líder e compressão. Hoje apresentamos o primeiro pedaço: a base de comunicação e de identidade sobre a qual todo o resto vai ser construído.

1.2 O que o Checkpoint 1 pedia

Sete itens:

- processo distribuído
- socket TCP
- cliente/servidor
- serialização
- framing de mensagens
- concorrência básica
- identificação de nós

E seis itens de verificação: conexão TCP funcionando, mensagem chega corretamente, header é interpretado, checksum é validado, dois processos podem conversar e não pode ocorrer segmentation fault.

Os sete itens estão implementados e os seis itens de verificação passam.

1.3 O que existe hoje

Dois programas:

- Super Peer — abre uma porta, fica escutando e guarda a lista de quem entrou.
- Peer — calcula a própria identidade, conecta no Super Peer e pede para entrar.

Cinco arquivos de código: network.c, protocol.c e peer.c (Aluno 1); node.c e superpeer.c (Aluno 2). Mais um Makefile, um README e uma pasta de testes.

O Makefile gera quatro executáveis. bin/superpeer e bin/peer são os programas; bin/node e bin/client são cópias deles, com os nomes que o script de teste procura. Vale saber de cor: bin/node é o Super Peer e bin/client é o Peer.

1.4 O caminho de uma mensagem, do início ao fim

Se for para guardar uma única coisa desta apresentação, é esta lista. Ela cobre a parte dos dois alunos e responde sozinha à maioria das perguntas possíveis.

1. O Peer lê o UUID do arquivo. Se o arquivo não existe, gera 16 bytes aleatórios de `/dev/urandom` e salva.
2. Calcula o NodeID: SHA-256 do IP, mais a porta, mais o UUID. Dá 32 bytes, mostrados como 64 caracteres em hexadecimal.
3. Conecta no Super Peer por TCP.
4. Monta a mensagem: um header de 99 bytes (versão, tipo, quem manda, para quem, id da transação, hora, tamanho do payload, checksum) mais o payload. No JOIN, o payload é o texto "127.0.0.1:9100", que é o endereço do próprio peer.
5. Calcula o CRC32 do pacote inteiro e grava no header.
6. Envia. O envio é em laço, porque o TCP pode aceitar só um pedaço de cada vez.
7. O Super Peer aceita a conexão e cria uma thread só para ela.
8. A thread lê exatamente 99 bytes de header. Olha o campo de tamanho do payload e lê exatamente aquele tanto. Isso é o framing.
9. Recalcula o CRC32 e compara com o que veio. Se não bate, descarta e fecha a conexão.
10. Vê que é um JOIN: confere a versão do protocolo, confere que o NodeID não é todo zero e lê o endereço do payload.
11. Insere na tabela de membros, protegida por mutex, e imprime a tabela.
12. Responde ACK. O Peer recebe e muda o estado para AUTHENTICATED.

1.5 Quem apresenta o quê

A regra do enunciado é que, no dia do checkpoint, apenas o aluno responsável pela sua parte apresenta.

- Aluno 1 — `network.c`, `protocol.c`, `peer.c`: `socket`, `bind`, `listen`, `accept`, `connect`, `send`, `recv`, `framing`, `header` e `checksum`.
- Aluno 2 — `node.c`, `superpeer.c`: `NodeID`, configuração, identificação do processo, registro do nó, tabela básica de membros e criação do Super Peer.

1.6 A demonstração, passo a passo

Passo 0 — compilar:

```
make
```

Avise antes que o build é pelo `make`, e não por `gcc *.c`. O motivo está no item 1.7.

Passo 1 — subir o Super Peer, no terminal 1:

```
./bin/superpeer superpeer 127.0.0.1 9000 sp.uuid
```

Aponte na tela: papel, IP, porta, estado e o NodeID de 64 caracteres. Essa é a identificação do processo, um dos itens pedidos.

Passo 2 — registrar um Peer, no terminal 2:

```
./bin/peer --cmd join --host 127.0.0.1 --port 9000 \  
--ip 127.0.0.1 --myport 9100 --uuid peer.uuid
```

No terminal 2 aparece RX ACK e Estado: AUTHENTICATED. No terminal 1 aparece RX JOIN com o NodeID de origem e a tabela de membros com uma linha. São dois processos conversando, com header interpretado e checksum validado — três itens de verificação de uma vez só.

Passo 3 — mostrar que a identidade não muda:

Rode o mesmo comando do Peer de novo e compare a linha NodeID. É igual, porque o UUID foi lido do arquivo. Aproveite para mostrar que a tabela continua com uma entrada só: JOIN repetido não duplica.

Passo 4 — sair da rede:

```
./bin/peer --cmd leave --host 127.0.0.1 --port 9000 \  
--ip 127.0.0.1 --myport 9100 --uuid peer.uuid
```

A tabela volta para zero membros.

Passo 5 — concorrência, se pedirem. Suba vários peers com portas e arquivos de uuid diferentes, ou rode o script de teste, que dispara 20 clientes ao mesmo tempo:

```
bash script_test.sh
```

Resultado esperado: PASS 10, FAIL 0.

Passo 6 — testes do protocolo, sem abrir socket nenhum:

```
make -C tests/c1 && ./tests/c1/test_protocol
```

Cinco casos: ida e volta, CRC detectando um byte trocado, buffer menor que o header, tamanho de payload inconsistente e payload vazio.

1.7 Três coisas que podem dar errado, com a resposta pronta

Se o professor rodar `gcc *.c` e não compilar. É verdade que não compila, e é de propósito. Cada binário é uma unidade de compilação só: `peer.c` e `superpeer.c` incluem `node.c` e `network.c`, e `network.c` inclui `protocol.c`. Juntar todos os `.c` num comando só dá função duplicada e dois `main`. O build é pelo `make`, e isso está explicado no README.

Se o `script_test.sh` falhar. São duas causas, nenhuma no código. A variável `ROOT` na linha 9 aponta para um caminho fixo da outra máquina e precisa ser ajustada. E o `shebang` está na linha 6 em vez da linha 1, então o script precisa ser rodado com `bash script_test.sh`, e não com `./script_test.sh`.

Se não gerar o log. A pasta `tests/c1/logs/` precisa existir; é para lá que o script manda a saída do servidor.

1.8 Uma pergunta para fazer ao professor

O enunciado se contradiz sobre quem apresenta Gossip, Heartbeat, detecção de falhas e eleição Bully. A tabela de divisão do trabalho (Seção 16.5) e o Checkpoint 6 colocam esses temas com o Aluno 2; os Checkpoints 3 e 4 colocam com o Aluno 1. Como só o responsável apresenta, vale confirmar isso hoje, antes que vire problema em 09/10.

Parte 2 — Sua parte: Aluno 1

Sua parte em uma frase: você fez o transporte. Como dois processos abrem uma conversa por TCP, e como uma mensagem vira bytes e volta a ser mensagem sem se perder no caminho.

Arquivos: `protocol.c`, `network.c` e `peer.c`. Itens do enunciado sob sua responsabilidade: criação do socket, `bind`, `listen`, `accept`, `connect`, `send`, `recv`, framing, header e checksum.

2.1 `protocol.c` — o formato da mensagem

O header é uma struct com 8 campos, na mesma ordem que o enunciado define na Seção 3.6: versão do protocolo, tipo da mensagem, nó de origem, nó de destino, id da transação, timestamp, tamanho do payload e checksum. No total, 99 bytes.

A struct é declarada `packed`. Sem isso, o compilador coloca bytes de enchimento entre os campos para alinhá-los na memória, e aí o tamanho da struct deixa de ser o tamanho do pacote na rede. `Packed` garante que os 99 bytes escritos são os 99 bytes enviados.

O enum tem os 20 tipos de mensagem da especificação, de `JOIN` e `LEAVE` até `ACK` e `ERROR`, mais `PING` e `PONG`, que usamos só como teste de vida. Neste checkpoint apenas seis são tratados; o resto está declarado para que os códigos não mudem de valor nos próximos checkpoints.

`empacotar_mensagem` aloca header mais payload, copia o header com o campo do checksum zerado, copia o payload logo atrás, calcula o CRC32 do bloco inteiro e grava o resultado no lugar do checksum.

`desempacotar_mensagem` confere que o tamanho recebido bate com header mais tamanho do payload, guarda o checksum que veio, zera o campo numa cópia, recalcula e compara. Se não bate, devolve erro e a mensagem é descartada.

`calcular_checksum` é um CRC32 padrão: polinômio refletido `0xEDB88320`, valor inicial com todos os bits em 1 e inversão dos bits no final.

Se perguntarem por que o checksum é zerado antes do cálculo: porque o campo faz parte do bloco que entra na conta. Com o valor dentro, quem recebe nunca reproduziria o mesmo número, porque precisaria já saber a resposta. Zerar dos dois lados é o que torna o cálculo repetível.

2.2 `network.c` — os sockets

`criar_servidor` faz `socket`, depois `SO_REUSEADDR`, depois `bind` e `listen` com fila de 10. O `SO_REUSEADDR` existe para dar para subir o servidor de novo logo após matar o processo, sem esperar o sistema liberar a porta.

`aceitar_cliente` faz o `accept` e devolve o descritor da conexão. `conectar_no_servidor` faz `socket`, converte o IP de texto para binário com `inet_pton` e faz `connect`.

As funções internas de enviar e de ler são as que fazem o framing funcionar: ficam em laço até completar o número exato de bytes pedido, porque `send` e `recv` podem transferir menos do que foi solicitado. O TCP não promete entregar tudo de uma vez.

`receber_mensagem` trabalha em duas etapas. Primeiro lê 99 bytes, que é um tamanho fixo e conhecido. Do header já lido, tira o tamanho do payload e lê exatamente aquele tanto. Só então entrega o bloco para a validação do checksum.

Resposta curta para "o que é framing": o TCP entrega um fluxo de bytes sem marcar onde uma mensagem acaba e a outra começa. Quem marca é o header de tamanho fixo, que carrega dentro dele o tamanho do resto.

2.3 peer.c — o cliente

Lê as opções da linha de comando: --cmd com join, leave ou ping; --host e --port do Super Peer; --ip e --myport do próprio peer; e --uuid com o arquivo de identidade.

Valida tudo antes de abrir qualquer socket: IP com inet_pton e portas na faixa de 1 a 65535. Argumento inválido imprime a causa e sai com código diferente de zero.

Monta a identidade chamando node_init, que é a parte do Aluno 2, imprime papel, IP, porta, estado e NodeID, e só então conecta.

Os estados acompanham a conexão: CONNECTING antes do connect, CONNECTED depois dele, e AUTHENTICATED quando o ACK do JOIN chega.

No JOIN, o payload é o texto ip:porta do próprio peer. É assim que o Super Peer aprende onde aquele nó pode ser encontrado depois.

Envia, espera a resposta, imprime RX seguido do tipo recebido, libera a memória da mensagem e fecha o socket.

2.4 Frases prontas

- O header tem 99 bytes e é a mesma estrutura da Seção 3.6 do enunciado.
- O framing está no laço de leitura: leio 99 bytes de header, tiro o tamanho do payload de dentro dele e leio exatamente aquele tanto.
- O checksum é um CRC32 sobre o pacote inteiro, com o próprio campo zerado nos dois lados.
- Todo malloc tem free, e o socket é fechado mesmo quando a mensagem é descartada por CRC inválido.

2.5 Se perguntarem dos pontos fracos

Reconheça. Saber onde o próprio código é frágil conta a favor.

- O tamanho do payload vem da rede sem limite máximo. Um valor muito grande faz o servidor tentar uma alocação enorme e, no extremo, estoura a soma de 32 bits com o tamanho do header. Falta um teto de payload e uma checagem da soma; é uma correção de poucas linhas.
- O header vai na ordem de bytes da máquina. Entre duas máquinas iguais funciona; entre arquiteturas diferentes, não. O enunciado pede big-endian para as comparações do Chord, então isso será ajustado no Checkpoint 3.
- A versão do protocolo só é conferida dentro do tratamento do JOIN, e não antes do desvio por tipo de mensagem.

Parte 3 — Perguntas que o professor pode fazer

Esta parte é igual nos dois roteiros. As respostas estão curtas de propósito: o objetivo é responder com segurança, não recitar. Onde o código tem uma limitação, a resposta assume a limitação — reconhecer um problema conhecido conta a favor, dizer que não existe conta contra.

Visão geral

1. Por que dois programas e não um só?

Porque os papéis são diferentes. O Super Peer escuta e guarda a lista de membros; o Peer conecta e se registra. Cada um tem seu próprio main. Nos próximos checkpoints haverá vários Super Peers conversando entre si.

2. O que é P2P híbrido?

Não é P2P puro, onde todos falam com todos, nem cliente-servidor, onde tudo passa por uma máquina só. Alguns nós têm mais responsabilidade, os Super Peers, e os Peers comuns se registram neles. Isso evita busca por inundação sem criar um ponto único de falha.

3. Me explica o que acontece quando um peer entra na rede.

Ele calcula o próprio NodeID a partir do IP, da porta e de um UUID salvo em disco. Conecta por TCP, monta uma mensagem JOIN com um header de 99 bytes e o próprio endereço no payload, calcula o CRC32 e envia. O Super Peer aceita, cria uma thread, lê o header, lê o payload pelo tamanho informado, confere o CRC, valida a versão e o NodeID, insere na tabela e responde ACK. O peer então passa para o estado AUTHENTICATED.

4. Onde está a serialização?

Em protocol.c, nas funções empacotar_mensagem e desempacotar_mensagem. Uma transforma a struct num vetor contínuo de bytes; a outra faz o caminho de volta, validando a integridade.

5. Onde está o framing e por que ele é necessário?

Em network.c. O TCP entrega um fluxo de bytes, não mensagens: ele não diz onde uma mensagem termina e a outra começa, e duas podem chegar coladas ou uma pode chegar partida. Resolvemos com um header de tamanho fixo, 99 bytes, que carrega dentro dele o tamanho do payload. Lemos 99, descobrimos o tamanho do resto e lemos exatamente aquele tanto.

6. Por que TCP e não UDP?

O enunciado pede TCP, e faz sentido: um registro de nó precisa chegar, na ordem e sem duplicar. Com UDP teríamos de reimplementar confirmação, reordenação e retransmissão por conta própria.

Protocolo e mensagem

7. Quantos bytes tem o header e quais são os campos?

99 bytes: versão do protocolo (1), tipo da mensagem (2), nó de origem (32), nó de destino (32), id da transação (16), timestamp (8), tamanho do payload (4) e checksum (4). É a mesma estrutura da Seção 3.6 do enunciado, na mesma ordem.

8. Por que a struct do header é packed?

Sem packed, o compilador insere bytes de enchimento entre os campos para alinhá-los na memória, e o tamanho da struct deixa de ser o tamanho do pacote na rede. Com packed, os 99 bytes que escrevemos são exatamente os 99 bytes que saem.

9. Por que CRC32 e não SHA-256 na mensagem?

CRC32 serve para detectar corrupção acidental de transmissão, que é o problema aqui, e é barato de calcular. O enunciado reserva o SHA-256 para a integridade do conteúdo dos arquivos, que entra nos próximos checkpoints.

10. Por que o campo do checksum é zerado antes de calcular?

Porque ele faz parte do bloco que entra na conta. Se o valor estivesse lá na hora do cálculo, quem recebe nunca conseguiria reproduzir o mesmo número, porque precisaria já saber a resposta. Zerando dos dois lados, o cálculo fica repetível.

11. E se eu mandar um pacote corrompido?

O CRC recalculado não bate, a função de desempacotar devolve erro, a mensagem é descartada e a conexão é fechada sem resposta. Temos um teste unitário que troca um byte do payload e confirma esse comportamento.

12. E se a versão do protocolo for diferente da esperada?

No JOIN, o Super Peer recusa e responde ERROR. Sendo honesto: hoje essa conferência está dentro do tratamento do JOIN, e não antes de decidir o que fazer com a mensagem. Um PING com versão errada e CRC correto ainda receberia PONG. Sabemos disso, e a correção é mover a checagem para antes do desvio.

13. Para que servem o id da transação e o nó de destino, se não são usados?

Estão no formato porque o enunciado define assim e porque os próximos checkpoints usam: o id da transação é do 2PC e o nó de destino é do roteamento pelo Chord. Hoje vão zerados nas requisições; nas respostas, o nó de destino já leva o NodeID de quem perguntou.

14. Por que existem PING e PONG, se não estão na lista do enunciado?

São apenas um teste de vida, usado pelo script de teste para saber se o servidor já subiu. Não fazem parte do protocolo do trabalho.

15. O enum tem 20 tipos de mensagem, mas vocês usam seis. Por quê?

O enum é a lista completa da Seção 5.1 do enunciado. Declaramos os códigos agora para que eles não mudem de valor depois, mas só implementamos o tratamento de JOIN, LEAVE, ACK, ERROR, PING e PONG, que é o escopo deste checkpoint.

Rede e concorrência

16. Como o servidor atende vários clientes ao mesmo tempo?

Uma thread por conexão. O laço principal só faz accept; quem atende é uma thread nova, criada com pthread_create e destacada com pthread_detach logo em seguida.

17. Por que thread por conexão e não select, epoll ou pool de threads?

Porque o volume é pequeno e a conexão é curta: uma mensagem, uma resposta e fecha. Thread por conexão é o modelo mais simples que resolve, e o enunciado pede concorrência básica. Pool ou epoll seria complexidade sem ganho neste ponto.

18. Por que pthread_detach?

Porque ninguém vai chamar pthread_join nessa thread. Sem o detach, os recursos dela ficariam ocupados até alguém recolher, e com muitas conexões isso vira vazamento.

19. Onde está o mutex e o que aconteceria sem ele?

Um único mutex protege toda leitura e escrita da tabela de membros. Sem ele, dois JOIN simultâneos poderiam escrever na mesma posição do vetor, ou atualizar o contador de membros um por cima do outro, e a tabela ficaria corrompida.

20. Vocês testaram concorrência de verdade?

Sim. O script de teste dispara 20 clientes ao mesmo tempo e confere que os 20 receberam resposta. Passa.

21. O laço no envio e na leitura é mesmo necessário?

É. O send pode transmitir menos bytes do que foi pedido, e o recv pode devolver menos do que foi pedido. Sem o laço, a mensagem sai ou entra pela metade.

Identidade e tabela de membros

22. O que é o NodeID e como é calculado?

É a identidade do nó no sistema: SHA-256 do IP, mais a porta, mais o UUID. Dá 32 bytes, que mostramos em 64 caracteres hexadecimais. A comparação entre dois NodeIDs é um memcmp sobre esses 32 bytes.

23. Por que o UUID é salvo em arquivo?

Para o nó continuar sendo o mesmo nó depois de reiniciar. O NodeID define a posição do nó no anel do Chord e a prioridade dele na eleição Bully. Se o UUID fosse sorteado a cada execução, o NodeID mudaria junto, um nó que reinicia voltaria como um nó completamente diferente, a finger table ficaria inválida e o teste do Checkpoint 5 não teria como passar.

24. Dá para mostrar que o NodeID não muda?

Dá. É só rodar o mesmo comando duas vezes e comparar a linha NodeID na tela. Apagando o arquivo .uuid, aí sim a identidade muda.

25. Dois nós podem ter o mesmo NodeID?

Só se tiverem o mesmo IP, a mesma porta e o mesmo UUID, o que na prática é o mesmo nó. O UUID são 16 bytes aleatórios, então a chance de colisão é desprezível.

26. O que significa o estado AUTHENTICATED? Vocês implementaram autenticação?

Não. É um handshake trivial: a versão do protocolo bateu e o NodeID é válido. Não tem senha, nem TLS, nem troca de chaves. O próprio enunciado diz que autenticação está fora do escopo e que o estado deve ser implementado como uma transição simples.

27. Por que vetor fixo de 64 e não lista ligada ou hash table?

Porque 64 membros é pouco e a busca linear não é gargalo. Vetor fixo não aloca memória dinâmica, então não vaza e não há ponteiro para errar. O enunciado pede a tabela básica; complicar a estrutura não traria nada em troca neste checkpoint.

28. E se a tabela encher?

A inserção devolve erro e o nó recebe ERROR. Em nenhuma situação escrevemos fora do vetor.

29. O que acontece se o mesmo peer mandar JOIN duas vezes?

Responde ACK e a tabela continua com uma entrada só. Tratamos como idempotente: o nó pediu para entrar e ele está dentro, então a resposta é positiva. Foi uma escolha nossa e está registrada na documentação.

30. E se chegar um tipo de mensagem que vocês não tratam?

Responde ERROR e fecha a conexão. Neste checkpoint só JOIN, LEAVE e PING têm tratamento.

31. Como o Super Peer descobre o endereço do peer?

Vem no payload do JOIN, no formato ip:porta. O Super Peer valida esse texto antes de guardar: se não for um IP válido com uma porta válida, recusa com ERROR.

32. Para que servem os campos `last_heartbeat` e `version` na tabela?

São campos que o enunciado define e que os próximos checkpoints usam. Hoje o `last_heartbeat` é preenchido com a hora do registro e nunca mais atualizado, e a `version` começa em 1. O uso de verdade vem com o heartbeat, no Checkpoint 3.

Robustez e testes

33. Como vocês testaram?

Em três níveis. Testes unitários do protocolo, sem abrir socket. Teste manual com dois terminais, um Super Peer e um Peer. E o script de teste do checkpoint, com 10 verificações, que passa 10 de 10.

34. Quais são os cinco testes unitários?

Ida e volta, onde a mensagem empacotada e desempacotada volta igual. CRC pegando um byte trocado. Buffer menor que o header. Header prometendo mais payload do que existe no buffer. E mensagem sem payload, que precisa voltar com o ponteiro nulo.

35. Compilaram com sanitizer?

Sim, com `-fsanitize=address,undefined`. Compila sem nenhum aviso, mesmo com `-Wall -Wextra -Wpedantic`, e o ciclo de JOIN e LEAVE roda sem nenhum relatório do sanitizer.

36. O código tem vazamento de memória?

No caminho normal, não. Todo malloc tem free correspondente: o buffer do empacotamento é liberado depois do envio, o payload recebido é liberado no fim do atendimento, a struct passada para a thread é liberada logo na entrada dela, e todo socket aberto é fechado.

37. Ocorreu segmentation fault em algum momento?

Não nos caminhos que testamos. É critério de aceitação explícito do checkpoint, e foi por isso que compilamos com sanitizer em vez de confiar só na execução normal.

38. Há alguma fragilidade que vocês já conhecem?

Há três, e todas são correções pequenas. O tamanho do payload vem da rede sem limite máximo, então um cliente mal-intencionado pode pedir uma alocação enorme. A versão do protocolo só é conferida dentro do JOIN, e não antes do desvio por tipo. E o header vai na ordem de bytes da máquina, o que funciona entre máquinas iguais, mas precisa virar big-endian antes do Chord, no Checkpoint 3.

39. Por que gcc *.c não compila o projeto de vocês?

Porque cada binário é uma unidade de compilação só: `peer.c` e `superpeer.c` incluem `node.c` e `network.c`, e `network.c` inclui `protocol.c`. Juntar todos os `.c` num comando dá função duplicada e dois main. O build correto é o make, e isso está documentado no README.

40. Por que fizeram assim, em vez de headers e linkagem normal?

Foi a forma mais direta de unir as duas partes sem um aluno ter de reescrever os arquivos do outro. Cada programa vira um objeto só, o Makefile cuida das dependências e não há risco de as duas metades saírem de sincronia. Se for preferível a separação clássica em `.h` com linkagem, a refatoração é pequena.

O que vem depois

41. O que falta para o sistema ficar completo?

Tudo dos checkpoints 2 a 6: metadados e fragmentação, DHT Chord com finger table, gossip e heartbeat, eleição Bully, replicação, 2PC, transferência de estado, cache LFU e compressão LZ4.

42. O que vem no Checkpoint 2?

Metadados dos arquivos, fragmentação em pedaços de 4 MiB, SHA-256 do arquivo inteiro para gerar o ObjectID, e upload e download pelo nome do arquivo.

43. E se o Super Peer cair?

Hoje os peers simplesmente não conseguem se registrar, e não há nada que perceba a queda. Detecção de falha e eleição de um novo coordenador são os Checkpoints 3 e 4.

44. Um nó que morre sem mandar LEAVE sai da tabela?

Não, hoje ele fica lá. É exatamente o problema que o heartbeat e o gossip do Checkpoint 3 resolvem: sem notícia por 15 segundos, o nó é considerado suspeito.

45. Usaram alguma biblioteca externa?

Só a libcrypto do OpenSSL, e apenas para a função SHA256. O resto é biblioteca padrão de C, POSIX e pthreads. A LZ4 entra a partir do Checkpoint 2.